has already received scrutiny from the engineering community and is a proven solution. The key to using patterns well is to thoroughly understand the problem space: a well-defined problem contains the seeds of its own solution, which is why time spent on planning and analysis is vital to creating good software.

This section is not intended as an exhaustive resource for all things concerning design patterns but should serve as a high-level introduction to the topic. Recommended further reading includes the two books already mentioned, and for PHP in particular there is *PHP 8 Objects, Patterns, and Practice* by my friend (and technical editor of this book!) Matt Zandstra, available from Apress Media. Now, without further ado, let us dive in.

Object-oriented design patterns are usually documented as UML class diagrams, which we have already seen earlier in the chapter, together with a description of the problems and solutions that the pattern is intended to address. The pattern might be a single class or a collection of classes related by inheritance or (especially) by interface dependencies and can be broadly sorted into the following types:

- Creational patterns
- Structural patterns
- Behavioral patterns

Creational Patterns

The process of creating new objects is straightforward; PHP's `new` keyword combined with a class name and parentheses will instantiate and return a reference to the new object, as described in Chapter 6, "Object Fundamentals." However, peppering your code with new objects can quickly lead to problems at runtime, and it is generally advisable to wrap object creation in code that is dedicated to managing the process. One venerable pattern intended to contain the (sometimes messy) business of making decisions around what to create (and how to create it) is the *Factory Method*. The idea behind the Factory Method is that as an application grows, classes often diversify into specialized sub-classes, and code that creates generalized objects representing products such as, say, physical books, might in the future need to create more specialized products, such as magazines or comics. As the new requirements pour in, your code can become a mess of conditional statements creating `new Book()`, or `new Magazine()`, and so on.